Fejlesztő adatai:

- Név: Ballai Péter Lóránt
- Neptun kód: EYSEED

**1. Feladatleírás**

Készítsünk programot, amellyel a következő kétszemélyes játékot lehet játszani. Adott egy n * n mezőből álló tábla, amelyen két játékos űrhajói helyezkednek el, középen pedig egy fekete lyuk. A játékos n -1 űrhajóval rendelkezik, amelyek átlóban helyezkednek el a táblán (az azonos színűek egymás mellett, ugyanazon az oldalon). A játékosok felváltva léphetnek. Az űrhajók vízszintesen, illetve függőlegesen mozoghatnak a táblán, de a fekete lyuk megzavarja a navigációjukat, így nem egy mezőt lépnek, hanem egészen addig haladnak a megadott irányba, amíg a tábla széle, a fekete lyuk, vagy egy másik, előtte lévő űrhajó meg nem állítja őket (tehát másik űrhajót átlépni nem lehet). Az a játékos győz, akinek sikerül űrhajóinak felét eljuttatnia a fekete lyukba. A program biztosítson lehetőséget új játék kezdésére a pályaméret megadásával (5*5, 7*7, 8*8), valamint játék mentésére és betöltésére. Ismerje fel, ha vége a játéknak, és jelenítse meg, melyik játékos győzött.

2. **UML Felhasználói Esetek (Use Case)**

**A feladat elemzése:** A játékot egy 5*5, 7*7 vagy 9*9 méretű négyzetrácsos táblán játssza két játékos (1. játékos – piros, 2. játékos – kék). Kezdetben a tábla középső mezőjén található a fekete lyuk. A főátlóban és a mellékátlóban helyezkednek el az űrhajók: a középső sor feletti átlós mezőkön az 1. játékos, a középső sor alatti átlós mezőkön pedig a 2. játékos n-1 darab űrhajója áll. A győzelemhez szükséges beküldött űrhajók száma (n-1) / 2.

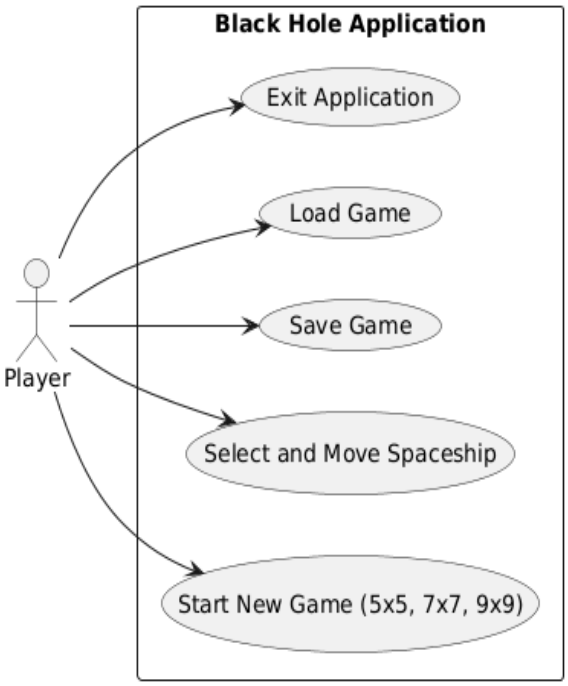
A soron következő játékos kiválasztja az egyik saját űrhajóját, majd megadja a mozgás irányát egérkattintással egy azonos sorban/oszlopban lévő mezőre, vagy a billentyűzet segítségével. Az űrhajó a kiválasztott irányba csúszik mindaddig, amíg:

- el nem éri a játéktábla szélét (ekkor a szélső mezőn megáll),
- egy másik űrhajóba nem ütközik (ekkor közvetlenül a másik űrhajó előtti mezőn megáll),
- bele nem csúszik a fekete lyukba (ekkor az űrhajó eltűnik a tábláról, és a játékos pontszáma eggyel nő).

Érvényes lépésnek csak az számít, ha az űrhajó ténylegesen elmozdul a kiindulási helyéről. Minden érvényes lépés után a másik játékos következik, kivéve, ha az adott lépéssel a játékos elérte a győzelemhez szükséges pontszámot.

**Felhasználói esetek:**

- **Új játék indítása (5*5, 7*7, 9*9):** A felhasználó a menüből kiválasztja a kívánt táblaméretet, mire a program alaphelyzetbe állítja a táblát, a pontszámokat és az ablakméretet.
- **Űrhajó kiválasztása és mozgatása:** A soron lévő játékos kijelöli saját űrhajóját, majd egy vízszintes vagy függőleges irány megadásával elcsúsztatja azt.
- **Játékállás mentése:** A felhasználó fájlba mentheti az aktuális táblaméretet, a soron lévő játékost, a fekete lyukba jutott hajók számát és a tábla mezőinek állapotát.
- **Játékállás betöltése:** A felhasználó egy korábban elmentett fájlból visszaállíthatja a játékállást.
- **Kilépés:** A felhasználó bezárja az alkalmazást.



3. **UML Osztálydiagram (Class Diagram)**
Az alkalmazás háromrétegű architektúrában épül fel: megjelenítés (BlackHole), üzleti logika (BlackHole.Model) és perzisztencia (BlackHole.Model adatelérési interfész és megvalósítás).

Perzisztencia réteg

- **IBlackHoleDataAccess (interfész):** Meghatározza a játékállás aszinkron betöltéséhez (LoadAsync) és mentéséhez (SaveAsync) szükséges műveleteket. Ez biztosítja, hogy az üzleti logika független maradjon a konkrét fájlkezeléstől, és egységtesztekben szimulálható (mockolható) legyen.
- **BlackHoleDataAccess (osztály):** Megvalósítja az IBlackHoleDataAccess interfészt szöveges állományok (StreamReader, StreamWriter) használatával. A fájl első sora a tábla méretét, az aktuális játékost és a két játékos pontszámát tartalmazza, a további sorok pedig a tábla mezőinek értékeit tárolják szóközzel elválasztva. Hibás fájl esetén IOException kivételt dob.

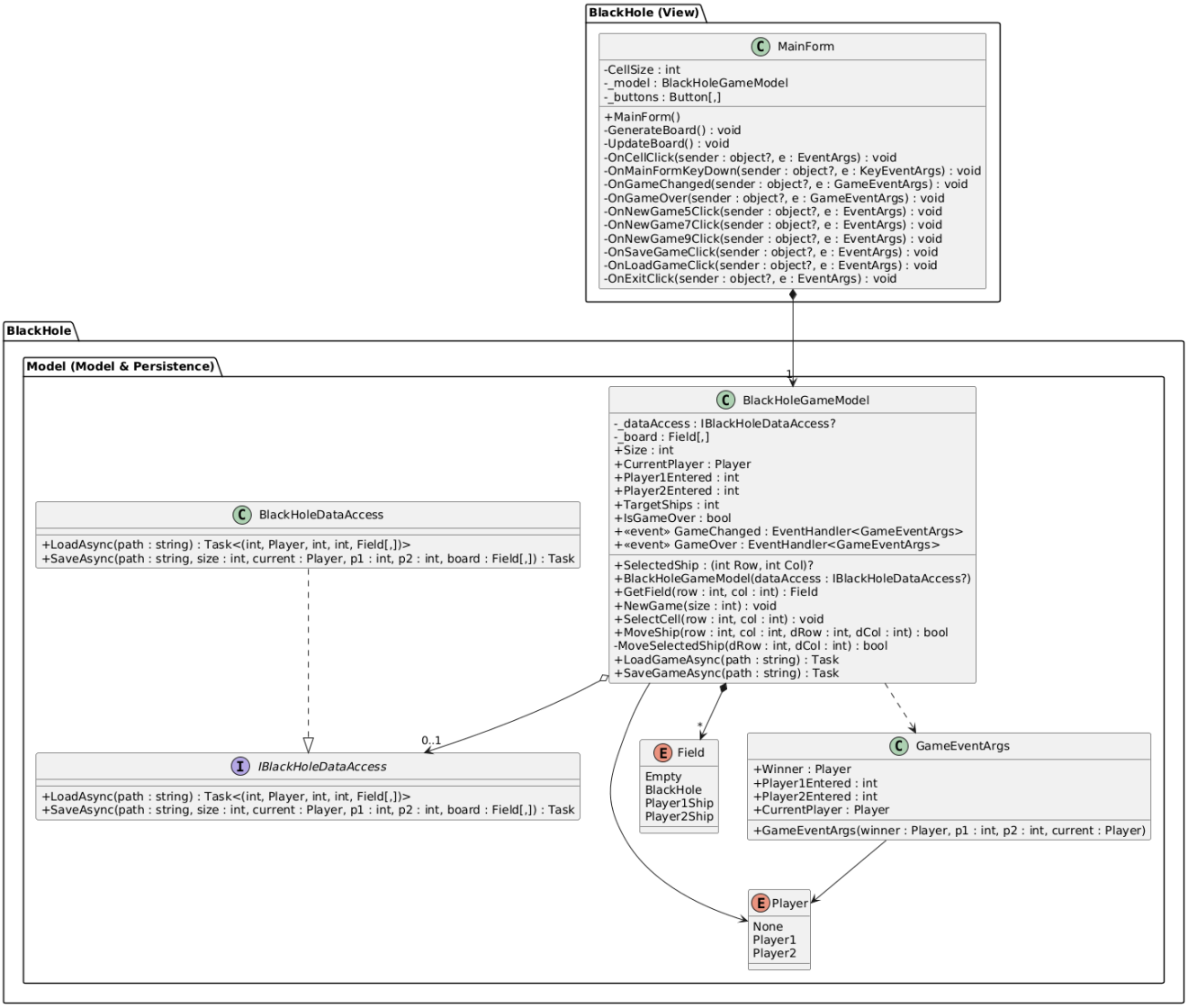
Modell réteg (BlackHole.Model)

- **Player (felsorolási típus):** A játékosokat reprezentálja (None, Player1, Player2).
- **Field (felsorolási típus):** Egy mező lehetséges állapotait írja le (Empty, BlackHole, Player1Ship, Player2Ship).
- **GameEventArgs (osztály):** Az EventArgs leszármazottja; a modell eseményeihez csomagolja a győztest (Winner), a fekete lyukba bejutott űrhajók számát (Player1Entered, Player2Entered), valamint a soron következő játékost (CurrentPlayer).
- **BlackHoleGameModel (osztály):** A játék fő üzleti logikáját megvalósító osztály.
- Tárolja a játéktáblát (_board), a tábla méretét (Size), a soron következő játékost (CurrentPlayer), a pontszámokat (Player1Entered, Player2Entered), a kijelölt űrhajó koordinátáit (SelectedShip) és a játék végét jelző állapotot (IsGameOver).
- A NewGame(int size) metódus inicializálja a táblát a megadott méretre, elhelyezi középen a fekete lyukat, az átlókban pedig a két játékos űrhajóit.
- A SelectCell(int row, int col) és MoveShip(int row, int col, int dRow, int dCol) metódusok felelősek a bábuk kijelöléséért és a csúszási logika (MoveSelectedShip) végrehajtásáért.
- Minden állapotváltozáskor kiváltja a GameChanged eseményt, győzelem esetén pedig a GameOver eseményt, amelyekre a nézet feliratkozik.

Nézet réteg (BlackHole)

- **MainForm (osztály):** A Windows Forms főablak, amely megjeleníti a menüsort (MenuStrip), az állapotsort (StatusStrip) és a dinamikusan generált nyomógombokból (Button[,]) álló játéktáblát (_boardPanel).

- Feliratkozik a modell GameChanged és GameOver eseményeire. A táblaméret változásakor (GenerateBoard) futási időben építi fel a gombmátrixot, és az ablak méretét (ClientSize) automatikusan a játéktábla méretéhez igazítja.
- Kezeli a felhasználói interakciókat: az egérkattintásokat (OnCellClick), a billentyűzet-leütéseket (OnMainFormKeyDown), valamint az új játék indítását, mentést és betöltést (SaveFileDialog, OpenFileDialog)



### 4. Tesztesetek

A modell réteg működését automatizált MSTest egységtesztek ellenőrzik a Test1 osztályban. A fájlrendszer-független teszteléshez a MockDataAccess belső segédosztály szimulálja az IBlackHoleDataAccess interfészt.

1. **NewGame_InitializesBoardCorrectly:** Paraméterezett teszt, amely ellenőrzi, hogy új játék indításakor a tábla mérete, a győzelmi cél, a kezdőjátékos, a fekete lyuk középső pozíciója és a két játékos n-1 darab űrhajójának száma megfelelően inicializálódik-e.
2. **MoveShip_SlidesUntilWallOrOtherShip:** Ellenőrzi, hogy egy űrhajó mozgatásakor a bábu nem csak egy mezőt lép, hanem egészen a következő űrhajó előtti mezőig vagy a falig csúszik, majd a kör átadódik a másik játékosnak.
3. **MoveShip_EntersBlackHole_IncrementsScore:** Ellenőrzi, hogy amikor egy űrhajó a középső fekete lyukba csúszik, a bábu eltűnik a tábláról, a fekete lyuk a helyén marad, és az adott játékos pontszáma (Player1Entered) eggyel növekszik.
4. **MoveShip_InvalidMove_ReturnsFalseAndDoesNotChangeTurn:** Ellenőrzi a szabálytalan lépések kezelését: ha a játékos a pályán kívülre próbál lépni (falnak ütközve helyben maradna), vagy az ellenfél űrhajóját próbálja mozgatni, a lépés sikertelen (false), és nem változik a soron lévő játékos.
5. **SaveAndLoadGame_UsesDataAccessCorrectly:** A MockDataAccess segítségével ellenőrzi, hogy a mentés (SaveGameAsync) pontosan átadja-e a modell állapotát a perzisztenciának, a betöltés (LoadGameAsync) pedig megfelelően visszaállítja-e a tábla méretét, a mezőket, a pontszámokat és az aktuális játékost.